

Final Project - Dungeon Crawler

COMP301

Dungeon Crawler - Part 2

Introduction

Welcome to the last part of the last assignment in COMP301! Dungeon Crawler - Part 2 will focus on building the controller and user interface in JavaFX. By the end of these instructions, you will have a fully functional project to show off.

What You Built

DO NOT MOVE ON FROM PART 1 UNTIL YOU ARE FINISHED

- (Part 1) Model:
Develop the core game logic and data structures. This layer will manage the dungeon's layout, rooms, the hero, monsters, items, and other game elements. The model is also responsible for maintaining game state and notifying observers about changes.

What You Will Build

- (Part 2) Controller:
Implement the controller to act as a mediator between the view and the model. It will handle user input (such as keyboard or mouse events), translate these events into actions in the game, and update the model accordingly. This design keeps the user interface decoupled from the business logic.
- (Part 2) View (JavaFX):
Create a dynamic user interface with JavaFX that visually represents the dungeon, its inhabitants, and interactions. The view should subscribe to updates from the model, using the Observer pattern to automatically refresh the display when the game state changes.

Apprentice: Controller

Controller

Now that you've built a fully functional game backend, it's time to connect it to the rest of the application. In Part 2, you'll implement the controller that sets up and interprets user actions (like arrow key presses or button clicks) and communicates with the model.

The controller's job is to act as the **bridge between the view and the model**. The View should *just* display things. The model should *just* hold persistent information. The controller does everything else. It exposes a simplified API of user-facing actions, such as "move left" or "start game," which the view can call directly. This keeps responsibilities cleanly separated and allows for a more testable and flexible architecture.

You will implement a class called `ControllerImpl` in the `[prefix].controller` package. This controller will handle user actions and translate them into model updates.

There should be new package in your project called `controller` at the same level as `model`.

The `controller` should have one method for every possible action that a user could take in your game. In proper MVC architecture, the View can *read* from the `model` but it cannot *modify* the Model. That is the job of the controller. Take a minute to write down all possible actions, your user could take. At a bare minimum, it should have:

- User can start the game
- User can move the hero up
- User can move the hero down
- User can move the hero left
- User can move the hero right

If you are doing any extra credit, your list may grow longer.
Now take a look at the **Controller** interface:

```
public interface Controller {
    public void moveUp();
    public void moveDown();
    public void moveLeft();
    public void moveRight();
    public void startGame();
}
```

Header and Field

Now fill in the details of the **ControllerImpl** class. The controller needs to implement **Controller** and encapsulate single instance variable: a reference to the **Model**. This variable should be declared **private final** to ensure it does not change after construction.

Constructor

- The constructor takes in an instance of the **Model** object and stores it.
- This allows the controller to access model methods and delegate actions.

Public Methods

These methods represent all the actions the user can perform:

- void moveUp()
- void moveDown()
- void moveLeft()
- void moveRight()
- void startGame()

Each method simply calls the corresponding method in the **model**. These controller methods will later be connected to user interface events like keyboard or button input.

You **should not** add any additional logic to these methods. Their sole purpose is to pass commands from the user interface into the model.

Enchanter: JavaFX

You are given a few additional starter files to help you develop the view:

- Main
- view.AppLauncher
- view.FXComponent
- view.View

Main

First, take a look at the `Main` class:

```
public class Main {
    public static void main(String[] args) {
        Application.launch(AppLauncher.class);
    }
}
```

This is the standard `main()` method for a JavaFX application. It simply delegates to `Application.launch()`, which starts the JavaFX runtime and hands off control to the `start(Stage stage)` method in `AppLauncher`.

AppLauncher

`AppLauncher` extends the `Application` class, which defines the lifecycle of a JavaFX app. The key method to override is `start(Stage stage)`, which is where you'll build the initial scene and display the user interface.

The `Stage` represents the top-level window in a JavaFX application. It is where scenes are displayed, and it's what ultimately appears on screen.

The class currently contains one line: `stage.show();`, which simply opens a blank window. You'll be adding more setup code here to configure the game window, initialize the model, and load the first view.

This is also where the `EventHandlers` for key presses will likely be implemented.

FXComponent

`FXComponent` defines a single method, `render()`, which returns a JavaFX `Parent` node. Any class that implements this interface can be rendered onto the stage.

```
public interface FXComponent {
    /** Render the component and return the resulting Parent object */
    Parent render();
}
```

This interface provides a consistent way to build reusable, modular UI components across your project.

View

The `View` class is currently a skeleton, but it's where you'll implement the graphical layout of your game. This class is responsible for rendering the current game state and updating the display when the model changes.

The starter `View` class implements both the `Observer` and `FXComponent` interfaces. `Observer` allows it to receive updates when the model changes (following the Observer pattern), and `FXComponent` allows it to produce JavaFX UI elements via `render()`.

This dual role allows `View` to bridge the model and the GUI — it listens for updates and responds by updating the interface.

To run the starter app, go to the Maven toolbar (right panel) → Plugins → `javafx` → `javafx:run`. You should see a blank white screen pop up.

To get your game to display something on the screen, you'll need to complete two main tasks:

1. Set up the Model-View-Controller (MVC) architecture.
2. Implement the `render()` method in your `View` class to create a visible interface.

Setting up the MVC architecture

Remember that the **Model** is the **Subject** and the **View** is an **Observer**. The **Model** keeps a reference to its **observers**, the **Controller** keeps a reference to the **Model**, and the **View** has references to both the **Model** and the **Controller**.

In App launcher, we need to create the MVC pattern.

First things first, set the window title to "[your name]'s Dungeon Crawler" using the `stage.setTitle()` method.

Next, set up the architecture by following the following steps:

1. Create the **model** by instantiating a new **Model**, providing it with the dimensions of your board. *This object will manage the game state and logic.*
2. Construct the **controller** by passing the model to a new **ControllerImpl**. *The controller needs access to the model so it can translate user actions into game behavior.*
3. Write the **View's** constructor such that it can take in a reference to the controller and model. Once you are ready instantiate the **view** in **AppLauncher** using `new View(pc, model)` *The view needs the controller to send input commands, the model to display current state, and the stage and dimensions to build and manage the JavaFX layout.*
4. Finally, register the **view** as an **observer** of the **model** using `model.addObserver(view)`. This ensures that the view is automatically notified and refreshed whenever the model changes, completing the Observer pattern integration.

Creating and Displaying the Scene

After setting up the MVC architecture, you need to render the initial user interface and display it in the application window. This involves creating a **Scene**, styling it, and attaching it to the JavaFX **Stage**.

To do this, you must:

1. Create a new **Scene** using `new Scene(view.render(), width, height)`. For now, pick whatever numbers you like for **width** and **height**. It may change once you render a gameboard and decide if the window is too big or too small by default. *The **render()** method of the view will eventually return the root JavaFX node that represents your UI layout.*
2. Add a stylesheet to your scene using `scene.getStylesheets().add("dungeon.css");`. *This allows you to customize the appearance of your game interface with CSS. This file should live under the resources folder in `src/main/resources/...`*
3. Attach the scene to the stage by calling `stage.setScene(scene);`. *This tells JavaFX what content should be displayed in the window.*
4. Finally, call `stage.show();` to make the application window visible to the user.

At this point, your game window should appear on screen, showing whatever components you've implemented in the `render()` method of your **View**. The starter code has a label that reads "Hello, world".

Sorcerer - The User Interface.

By now you have everything you need to build the interface for your dungeon crawler. From here on out, the instructions will be a little more vague, because you have freedom to style this game according to your own preferences.

You will need at least two more **FXViewComponents**:

- **TitleScreenView**: This is your starting screen before beginning a new game, and you will return to it after you die. Whenever the model shows that the game is in its **END_GAME** state, this is the **View** that should render.
- **GameView**: This is your **IN_PROGRESS View**.

You may choose to have other UI elements: Some examples:

- a text-based output panel for game events.
- a high score label
- an end game button
- toggle switches for different modes
- etc.

A complete list of JavaFX controls (Look for the **Controls** section) and their associated css tags can be found at:

<https://docs.oracle.com/javafx/2/api/javafx/scene/doc-files/cssref.html>

In addition, you will now have to implement the `update()` method to re-render the screen when the model notifies the View that something has changed.

update()

The `Model` will call `view`, whenever something has changed, which should trigger the `View` to rerender itself to show the updates. If you look back to `AppLauncher`, you rendered the `View` passing the results of `view.render()` to a scene that gets set on the stage. You need to do that again, but here in the `View` method. *Hint: You may choose to modify the constructor to add more arguments to give you access to the objects you need.*

```
Scene scene = new Scene(view.render(), width, height);
scene.getStylesheets().add("dungeon.css");
stage.setScene(scene);
```

TitleScreenView

`TitleScreenView` should have the following components:

- Label that displays the overall Highest Score.
- Label that displays the Last Score.
- Button that will start the game.
- (Optional Bonus Interactions)
- "By [your name]"

Required Styling:

- It should be styled in a way that looks like a title Screen, with the title of the game in big (bold) letters, centered horizontally in the window *Hint: remember StackPane.*
- All text should be clearly readable on the background with good contrasting colors. The text itself should be in a legible font, large enough that the vast majority of people should be able to read it without glasses.
- The button should look like a button.

GameView

`GameView` should have the following components:

- A `GridPane` representing the game board that renders correctly. *Hint: GridPane lays out items vertically, which means the traditional row/column order is swapped from a 2-D array*
- Control Buttons for the user to control the hero.
- A Label that displays the current score.

- Arrow Key Presses for WASD should work to move the hero around.
- (optional) High score
- (Optional) any other relevant information

Required Styling: This should look like an arcade game.

- Board, there should be no space between the different spaces so that it looks like one cohesive object.
- The control buttons should not be all in one line. They should be arranged according to their direction with up being above down, and left being to the left of right.

Merlin (Optional)

Extra credit! *(This is only available to students who have finished the rest of the game. Your game should be playable and styled before attempting. If these requirements are not sufficiently met, the TAs reserve the right to not grant extra credit. You can earn up to 30pts extra credit on this assignment that will go towards your Assignment Grade Total.*

Option 1: HardMode vs Easy Mode (up to 10pts)

Give Players a challenge! (Or a break).

Make two different enemy movement behaviors. The default one is random movements (implemented in Part 1), hardmode has the enemies that zero in on the hero. Every move, they try to move closer for a tasty snack. User should be able to toggle between modes (with a toggle JavaFX component) on the Title page before beginning a new game.

Option 2: Create a new Piece and game mechanic (up to 10pts)

Create a new **Piece** that adds new features to the game. To take advantage of this opportunity you must explain the mechanic in a **Text** component on the **GameView** page. It must be something that can come up in every room, and be easy to see, interact with, and replicate. This will be graded on mechanic complexity. (A new piece that just changes the score will get 2 points)

Option 3: Dungeons and Rainbows (up to 5 pts)

Design two aesthetics. This should carry through to the Piece assets as well. It should be two non sequitur themes like Bubblegum Beach Theme vs Zombie Apocalypse Theme. The game assets need to match the aesthetic, and there should be a toggle switch (with a toggle JavaFX component) on each page to switch back and forth between them. Each theme will be graded on the same elements of style and usability mentioned above.

Option 4: Add music! (Up to 5 points)

The pom.xml should already support this. The music should tie in with your game and help support the theme. If you are doing option 3, this should also change with your theme.

Option 5: Add in a timer! (Up to 5 points)

Add in an elapsed time timer on the GameView so that you can see how long you've survived. Timer should persist through multiple levels but reset on gameOver. The time should then be displayed next to your scores for last score and high score.

Option 6: Choose your own adventure (up to 10pts)

If you have a really great idea that is not covered in the other Extra Credit options, and you are absolutely dying to try it out, email Prof Prairie with a paragraph proposal and the subject "Bonus Game Mechanic Request", and wait for approval.